

```

d_hidden = error_hidden * sigmoid_derivative(hidden_output)

# ----- Update Weights and Biases -----
W2 += hidden_output.T.dot(d_output) * learning_rate
b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate

W1 += X.T.dot(d_hidden) * learning_rate
b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

if epoch % 1000 == 0:
    print(f"Epoch {epoch}, Loss: {loss:.6f}")

print("\nFinal Predictions:")
for i in range(len(X)):
    prediction = final_output[i][0]
    predicted_class = 1 if prediction >= 0.5 else 0
    print(f"Input: {X[i]} -> Predicted: {prediction:.4f} -> Class: {predicted_class}")

Epoch 0, Loss: 0.294351
Epoch 1000, Loss: 0.244410
Epoch 2000, Loss: 0.203539
Epoch 3000, Loss: 0.153349
Epoch 4000, Loss: 0.046331
Epoch 5000, Loss: 0.015615
Epoch 6000, Loss: 0.008448
Epoch 7000, Loss: 0.005614
Epoch 8000, Loss: 0.004146
Epoch 9000, Loss: 0.003264

Final Predictions:
Input: [0 0] -> Predicted: 0.0540 -> Class: 0
Input: [0 1] -> Predicted: 0.9505 -> Class: 1
Input: [1 0] -> Predicted: 0.9501 -> Class: 1
Input: [1 1] -> Predicted: 0.0536 -> Class: 0

```

LAB 7

```

# =====
# 1. Import Libraries
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

```

```

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# =====
# 2. Load Dataset as DataFrame
# =====
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# Print Columns
print("\nColumns in Dataset:")
print(df.columns)

# =====
# 3. Feature-Target Split
# =====
X = df.drop("Price", axis=1)
y = df["Price"]

# =====
# 4. Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Feature Scaling (VERY IMPORTANT)
# =====
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# =====
# 6. Build Multilayer Model
# =====
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu',
input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1) # Linear activation for regression
])

model.compile(

```

```

        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss='mse',
        metrics=['mae']
    )

model.summary()

# =====
# 7. Train Model
# =====
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# =====
# 8. Evaluate Model
# =====
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# =====
# 9. Plot Training Curve
# =====
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# =====
# 10. Actual vs Predicted Plot
# =====
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")

```

```
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

Dataset Shape: (20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup
Latitude \						
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556
37.88						
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842
37.86						
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260
37.85						
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945
37.85						
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467
37.85						

	Longitude	Price
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

Columns in Dataset:

```
Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population',
      'AveOccup',
      'Latitude', 'Longitude', 'Price'],
      dtype='object')
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

Model: "sequential_4"

Layer (type)	Output Shape
Param #	
dense_12 (Dense)	(None, 128)
1,152	

dense_13 (Dense)	(None, 64)	
8,256		
dense_14 (Dense)	(None, 32)	
2,080		
dense_15 (Dense)	(None, 1)	
33		

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

465/465 ————— 5s 6ms/step - loss: 0.6934 - mae: 0.5580
- val_loss: 0.4177 - val_mae: 0.4631

Epoch 2/100

465/465 ————— 3s 7ms/step - loss: 0.3722 - mae: 0.4323
- val_loss: 0.4420 - val_mae: 0.4955

Epoch 3/100

465/465 ————— 3s 5ms/step - loss: 0.3522 - mae: 0.4135
- val_loss: 0.3614 - val_mae: 0.4299

Epoch 4/100

465/465 ————— 4s 2ms/step - loss: 0.3318 - mae: 0.4021
- val_loss: 0.3689 - val_mae: 0.4453

Epoch 5/100

465/465 ————— 1s 2ms/step - loss: 0.3178 - mae: 0.3924
- val_loss: 0.3336 - val_mae: 0.4029

Epoch 6/100

465/465 ————— 1s 2ms/step - loss: 0.3063 - mae: 0.3843
- val_loss: 0.3334 - val_mae: 0.4033

Epoch 7/100

465/465 ————— 1s 2ms/step - loss: 0.2967 - mae: 0.3768
- val_loss: 0.3247 - val_mae: 0.3836

Epoch 8/100

465/465 ————— 1s 3ms/step - loss: 0.2885 - mae: 0.3703
- val_loss: 0.3100 - val_mae: 0.3806

Epoch 9/100

465/465 ————— 2s 4ms/step - loss: 0.2877 - mae: 0.3685
- val_loss: 0.3150 - val_mae: 0.3823

Epoch 10/100

465/465 ————— 2s 3ms/step - loss: 0.2838 - mae: 0.3652
- val_loss: 0.3145 - val_mae: 0.3891

Epoch 11/100

```
465/465 _____ 1s 3ms/step - loss: 0.2799 - mae: 0.3622
- val_loss: 0.3076 - val_mae: 0.3791
Epoch 12/100
465/465 _____ 1s 2ms/step - loss: 0.2775 - mae: 0.3608
- val_loss: 0.3020 - val_mae: 0.3699
Epoch 13/100
465/465 _____ 1s 2ms/step - loss: 0.2884 - mae: 0.3613
- val_loss: 0.3112 - val_mae: 0.3954
Epoch 14/100
465/465 _____ 1s 2ms/step - loss: 0.2714 - mae: 0.3553
- val_loss: 0.3035 - val_mae: 0.3829
Epoch 15/100
465/465 _____ 1s 2ms/step - loss: 0.2718 - mae: 0.3551
- val_loss: 0.3168 - val_mae: 0.3735
Epoch 16/100
465/465 _____ 1s 2ms/step - loss: 0.2660 - mae: 0.3510
- val_loss: 0.2952 - val_mae: 0.3748
Epoch 17/100
465/465 _____ 1s 2ms/step - loss: 0.2606 - mae: 0.3465
- val_loss: 0.2978 - val_mae: 0.3834
Epoch 18/100
465/465 _____ 2s 4ms/step - loss: 0.2602 - mae: 0.3473
- val_loss: 0.3271 - val_mae: 0.4021
Epoch 19/100
465/465 _____ 2s 4ms/step - loss: 0.2589 - mae: 0.3465
- val_loss: 0.2950 - val_mae: 0.3788
Epoch 20/100
465/465 _____ 2s 3ms/step - loss: 0.2548 - mae: 0.3437
- val_loss: 0.2952 - val_mae: 0.3656
Epoch 21/100
465/465 _____ 2s 3ms/step - loss: 0.2561 - mae: 0.3466
- val_loss: 0.2928 - val_mae: 0.3571
Epoch 22/100
465/465 _____ 1s 3ms/step - loss: 0.2533 - mae: 0.3414
- val_loss: 0.3070 - val_mae: 0.3825
Epoch 23/100
465/465 _____ 1s 2ms/step - loss: 0.2489 - mae: 0.3404
- val_loss: 0.2921 - val_mae: 0.3738
Epoch 24/100
465/465 _____ 1s 3ms/step - loss: 0.2473 - mae: 0.3365
- val_loss: 0.2878 - val_mae: 0.3596
Epoch 25/100
465/465 _____ 1s 2ms/step - loss: 0.2474 - mae: 0.3384
- val_loss: 0.2893 - val_mae: 0.3669
Epoch 26/100
465/465 _____ 1s 2ms/step - loss: 0.2432 - mae: 0.3360
- val_loss: 0.2874 - val_mae: 0.3690
Epoch 27/100
465/465 _____ 2s 4ms/step - loss: 0.2421 - mae: 0.3339
```

```
- val_loss: 0.2960 - val_mae: 0.3686
Epoch 28/100
465/465 _____ 2s 4ms/step - loss: 0.2411 - mae: 0.3339
- val_loss: 0.2758 - val_mae: 0.3558
Epoch 29/100
465/465 _____ 1s 3ms/step - loss: 0.2381 - mae: 0.3322
- val_loss: 0.2776 - val_mae: 0.3515
Epoch 30/100
465/465 _____ 1s 2ms/step - loss: 0.2392 - mae: 0.3327
- val_loss: 0.2780 - val_mae: 0.3551
Epoch 31/100
465/465 _____ 1s 2ms/step - loss: 0.2352 - mae: 0.3299
- val_loss: 0.2849 - val_mae: 0.3545
Epoch 32/100
465/465 _____ 1s 2ms/step - loss: 0.2416 - mae: 0.3300
- val_loss: 0.2813 - val_mae: 0.3551
Epoch 33/100
465/465 _____ 1s 2ms/step - loss: 0.2345 - mae: 0.3292
- val_loss: 0.2805 - val_mae: 0.3610
Epoch 34/100
465/465 _____ 1s 2ms/step - loss: 0.2303 - mae: 0.3265
- val_loss: 0.2858 - val_mae: 0.3640
Epoch 35/100
465/465 _____ 1s 2ms/step - loss: 0.2279 - mae: 0.3238
- val_loss: 0.2846 - val_mae: 0.3545
Epoch 36/100
465/465 _____ 1s 2ms/step - loss: 0.2299 - mae: 0.3253
- val_loss: 0.2843 - val_mae: 0.3759
Epoch 37/100
465/465 _____ 2s 3ms/step - loss: 0.2282 - mae: 0.3243
- val_loss: 0.2821 - val_mae: 0.3650
Epoch 38/100
465/465 _____ 2s 2ms/step - loss: 0.2236 - mae: 0.3222
- val_loss: 0.2776 - val_mae: 0.3641
Epoch 39/100
465/465 _____ 1s 2ms/step - loss: 0.2241 - mae: 0.3229
- val_loss: 0.2840 - val_mae: 0.3655
Epoch 40/100
465/465 _____ 1s 2ms/step - loss: 0.2237 - mae: 0.3217
- val_loss: 0.2760 - val_mae: 0.3624
Epoch 41/100
465/465 _____ 1s 2ms/step - loss: 0.2214 - mae: 0.3204
- val_loss: 0.2754 - val_mae: 0.3558
Epoch 42/100
465/465 _____ 1s 2ms/step - loss: 0.2206 - mae: 0.3190
- val_loss: 0.2867 - val_mae: 0.3556
Epoch 43/100
465/465 _____ 1s 2ms/step - loss: 0.2169 - mae: 0.3180
- val_loss: 0.2810 - val_mae: 0.3505
```

```
Epoch 44/100
465/465 _____ 1s 2ms/step - loss: 0.2207 - mae: 0.3189
- val_loss: 0.2697 - val_mae: 0.3504
Epoch 45/100
465/465 _____ 1s 2ms/step - loss: 0.2149 - mae: 0.3170
- val_loss: 0.2711 - val_mae: 0.3421
Epoch 46/100
465/465 _____ 1s 2ms/step - loss: 0.2128 - mae: 0.3139
- val_loss: 0.2706 - val_mae: 0.3520
Epoch 47/100
465/465 _____ 2s 4ms/step - loss: 0.2117 - mae: 0.3149
- val_loss: 0.2718 - val_mae: 0.3476
Epoch 48/100
465/465 _____ 2s 2ms/step - loss: 0.2121 - mae: 0.3144
- val_loss: 0.2804 - val_mae: 0.3566
Epoch 49/100
465/465 _____ 1s 2ms/step - loss: 0.2171 - mae: 0.3156
- val_loss: 0.2771 - val_mae: 0.3594
Epoch 50/100
465/465 _____ 1s 2ms/step - loss: 0.2107 - mae: 0.3140
- val_loss: 0.2694 - val_mae: 0.3606
Epoch 51/100
465/465 _____ 1s 2ms/step - loss: 0.2065 - mae: 0.3107
- val_loss: 0.2633 - val_mae: 0.3532
Epoch 52/100
465/465 _____ 1s 3ms/step - loss: 0.2043 - mae: 0.3092
- val_loss: 0.3089 - val_mae: 0.3740
Epoch 53/100
465/465 _____ 1s 3ms/step - loss: 0.2067 - mae: 0.3118
- val_loss: 0.2717 - val_mae: 0.3551
Epoch 54/100
465/465 _____ 1s 3ms/step - loss: 0.2037 - mae: 0.3095
- val_loss: 0.2531 - val_mae: 0.3390
Epoch 55/100
465/465 _____ 2s 3ms/step - loss: 0.2034 - mae: 0.3085
- val_loss: 0.2675 - val_mae: 0.3478
Epoch 56/100
465/465 _____ 2s 4ms/step - loss: 0.2038 - mae: 0.3067
- val_loss: 0.2695 - val_mae: 0.3446
Epoch 57/100
465/465 _____ 1s 3ms/step - loss: 0.1985 - mae: 0.3053
- val_loss: 0.2590 - val_mae: 0.3454
Epoch 58/100
465/465 _____ 1s 3ms/step - loss: 0.2009 - mae: 0.3071
- val_loss: 0.2748 - val_mae: 0.3614
Epoch 59/100
465/465 _____ 1s 3ms/step - loss: 0.1972 - mae: 0.3051
- val_loss: 0.2629 - val_mae: 0.3440
Epoch 60/100
```


465/465 _____ 1s 2ms/step - loss: 0.1968 - mae: 0.3050
- val_loss: 0.2676 - val_mae: 0.3634
Epoch 61/100
465/465 _____ 1s 3ms/step - loss: 0.1950 - mae: 0.3025
- val_loss: 0.2653 - val_mae: 0.3479
Epoch 62/100
465/465 _____ 1s 3ms/step - loss: 0.1947 - mae: 0.3044
- val_loss: 0.2693 - val_mae: 0.3506
Epoch 63/100
465/465 _____ 1s 3ms/step - loss: 0.1945 - mae: 0.3037
- val_loss: 0.2639 - val_mae: 0.3496
Epoch 64/100
465/465 _____ 1s 3ms/step - loss: 0.1908 - mae: 0.3011
- val_loss: 0.2632 - val_mae: 0.3517
Epoch 65/100
465/465 _____ 2s 4ms/step - loss: 0.1928 - mae: 0.3019
- val_loss: 0.2809 - val_mae: 0.3491
Epoch 66/100
465/465 _____ 2s 3ms/step - loss: 0.1887 - mae: 0.2989
- val_loss: 0.2859 - val_mae: 0.3701
Epoch 67/100
465/465 _____ 1s 3ms/step - loss: 0.1893 - mae: 0.2998
- val_loss: 0.2626 - val_mae: 0.3477
Epoch 68/100
465/465 _____ 2s 4ms/step - loss: 0.1882 - mae: 0.2979
- val_loss: 0.2666 - val_mae: 0.3545
Epoch 69/100
465/465 _____ 2s 3ms/step - loss: 0.1871 - mae: 0.2973
- val_loss: 0.2713 - val_mae: 0.3539
Epoch 70/100
465/465 _____ 1s 2ms/step - loss: 0.1860 - mae: 0.2967
- val_loss: 0.2580 - val_mae: 0.3392
Epoch 71/100
465/465 _____ 1s 3ms/step - loss: 0.1843 - mae: 0.2957
- val_loss: 0.2796 - val_mae: 0.3555
Epoch 72/100
465/465 _____ 1s 3ms/step - loss: 0.1821 - mae: 0.2952
- val_loss: 0.2880 - val_mae: 0.3735
Epoch 73/100
465/465 _____ 3s 4ms/step - loss: 0.1870 - mae: 0.2975
- val_loss: 0.2753 - val_mae: 0.3476
Epoch 74/100
465/465 _____ 1s 3ms/step - loss: 0.1815 - mae: 0.2932
- val_loss: 0.2698 - val_mae: 0.3542
Epoch 75/100
465/465 _____ 1s 3ms/step - loss: 0.1816 - mae: 0.2936
- val_loss: 0.2808 - val_mae: 0.3632
Epoch 76/100
465/465 _____ 1s 3ms/step - loss: 0.1794 - mae: 0.2914

```
- val_loss: 0.2688 - val_mae: 0.3446
Epoch 77/100
465/465 _____ 1s 3ms/step - loss: 0.1779 - mae: 0.2912
- val_loss: 0.2676 - val_mae: 0.3435
Epoch 78/100
465/465 _____ 1s 3ms/step - loss: 0.1785 - mae: 0.2908
- val_loss: 0.2477 - val_mae: 0.3329
Epoch 79/100
465/465 _____ 1s 3ms/step - loss: 0.1764 - mae: 0.2896
- val_loss: 0.2705 - val_mae: 0.3471
Epoch 80/100
465/465 _____ 1s 3ms/step - loss: 0.1754 - mae: 0.2898
- val_loss: 0.2755 - val_mae: 0.3601
Epoch 81/100
465/465 _____ 2s 3ms/step - loss: 0.1762 - mae: 0.2897
- val_loss: 0.2580 - val_mae: 0.3387
Epoch 82/100
465/465 _____ 2s 5ms/step - loss: 0.1732 - mae: 0.2883
- val_loss: 0.2608 - val_mae: 0.3436
Epoch 83/100
465/465 _____ 1s 3ms/step - loss: 0.1741 - mae: 0.2877
- val_loss: 0.2809 - val_mae: 0.3701
Epoch 84/100
465/465 _____ 1s 3ms/step - loss: 0.1740 - mae: 0.2888
- val_loss: 0.2713 - val_mae: 0.3569
Epoch 85/100
465/465 _____ 1s 3ms/step - loss: 0.1715 - mae: 0.2877
- val_loss: 0.2616 - val_mae: 0.3522
Epoch 86/100
465/465 _____ 1s 3ms/step - loss: 0.1712 - mae: 0.2870
- val_loss: 0.2742 - val_mae: 0.3534
Epoch 87/100
465/465 _____ 1s 2ms/step - loss: 0.1682 - mae: 0.2844
- val_loss: 0.2574 - val_mae: 0.3432
Epoch 88/100
465/465 _____ 1s 3ms/step - loss: 0.1689 - mae: 0.2849
- val_loss: 0.2559 - val_mae: 0.3439
Epoch 89/100
465/465 _____ 1s 3ms/step - loss: 0.1702 - mae: 0.2864
- val_loss: 0.2669 - val_mae: 0.3534
Epoch 90/100
465/465 _____ 1s 3ms/step - loss: 0.1674 - mae: 0.2842
- val_loss: 0.2671 - val_mae: 0.3429
Epoch 91/100
465/465 _____ 2s 5ms/step - loss: 0.1693 - mae: 0.2831
- val_loss: 0.2614 - val_mae: 0.3405
Epoch 92/100
465/465 _____ 2s 4ms/step - loss: 0.1658 - mae: 0.2826
- val_loss: 0.2698 - val_mae: 0.3413
```

Epoch 93/100
465/465 _____ 3s 7ms/step - loss: 0.1652 - mae: 0.2805
- val_loss: 0.2712 - val_mae: 0.3550
Epoch 94/100
465/465 _____ 6s 8ms/step - loss: 0.1644 - mae: 0.2821
- val_loss: 0.2572 - val_mae: 0.3408
Epoch 95/100
465/465 _____ 4s 8ms/step - loss: 0.1634 - mae: 0.2803
- val_loss: 0.2621 - val_mae: 0.3409
Epoch 96/100
465/465 _____ 3s 5ms/step - loss: 0.1639 - mae: 0.2805
- val_loss: 0.2653 - val_mae: 0.3487
Epoch 97/100
465/465 _____ 2s 5ms/step - loss: 0.1627 - mae: 0.2805
- val_loss: 0.2796 - val_mae: 0.3534
Epoch 98/100
465/465 _____ 1s 3ms/step - loss: 0.1630 - mae: 0.2804
- val_loss: 0.2741 - val_mae: 0.3535
Epoch 99/100
465/465 _____ 1s 3ms/step - loss: 0.1608 - mae: 0.2787
- val_loss: 0.2650 - val_mae: 0.3425
Epoch 100/100
465/465 _____ 1s 3ms/step - loss: 0.1610 - mae: 0.2776
- val_loss: 0.2734 - val_mae: 0.3526
129/129 _____ 0s 2ms/step - loss: 0.2650 - mae: 0.3376
129/129 _____ 0s 2ms/step

Test MAE: 0.3376142978668213
RMSE: 0.5148124701167531
R2 Score: 0.7977487305196176

Training vs Validation Loss

